

You can think of lists, arrays, and strings as linear trees; the root is the first element, and the nodes have only one child node. This linear tree terminates at its one leaf node. These linear trees are called *linked lists*, as each node has only one “next” node until the end of the list. [Figure 4-3](#) shows a linked list that stores the characters in the word *HELLO*.



Figure 4-3: A linked list data structure storing HELLO. Linked lists can be considered a kind of tree data structure.

We’ll use the code for the tree in [Figure 4-2](#) for this chapter’s examples. A tree traversal algorithm will visit each of the nodes in a tree by following the edges, starting from a root node.

Traversing the Tree

We can write code to access data in any node by starting from the root node in `root`. For example, after entering the tree code into the Python or JavaScript interactive shell, run the following:

```
>>> root['children'][1]['data']  
'C'  
>>> root['children'][1]['children'][0]['data']  
'E'
```

Our tree traversal code can be written as a recursive function because tree data structures have a self-similar structure: a parent node has child nodes, and each child node is the parent node of its own children. Tree traversal algorithms ensure that your programs can access or modify the data in every node in the tree no matter its shape or size.

Let’s ask the three questions about recursive algorithms for our tree traversal code:

What is the base case? A leaf node, which has no more children and requires no more recursive calls, causing the algorithm to backtrack to a previous parent node.